

Article

Automatic Taxonomy Classification by Pretrained Language Model

Ayato Kuwana, Atsushi Oba, Ranto Sawai  and Incheon Paik *

Graduate Department of Computer Science and Information Systems, University of Aizu, Tsuruga, Fukushima 965-8580, Japan; m5241101@u-aizu.ac.jp (A.K.); happy.dememadara@gmail.com (A.O.); m5251141@u-aizu.ac.jp (R.S.)

* Correspondence: paikic@u-aizu.ac.jp

Abstract: In recent years, automatic ontology generation has received significant attention in information science as a means of systemizing vast amounts of online data. As our initial attempt of ontology generation with a neural network, we proposed a recurrent neural network-based method. However, updating the architecture is possible because of the development in natural language processing (NLP). By contrast, the transfer learning of language models trained by a large, unlabeled corpus has yielded a breakthrough in NLP. Inspired by these achievements, we propose a novel workflow for ontology generation comprising two-stage learning. Our results showed that our best method improved accuracy by over 12.5%. As an application example, we applied our model to the Stanford Question Answering Dataset to show ontology generation in a real field. The results showed that our model can generate a good ontology, with some exceptions in the real field, indicating future research directions to improve the quality.

Keywords: ontology; automation; natural language processing (NLP); pretrained model



Citation: Kuwana, A.; Oba, A.; Sawai, R.; Paik, I. Automatic Taxonomy Classification by Pretrained Language Model. *Electronics* **2021**, *10*, 2656. <https://doi.org/10.3390/electronics10212656>

Academic Editor: Luca Mesin

Received: 22 September 2021

Accepted: 26 October 2021

Published: 29 October 2021

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2021 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In recent years, the Internet has yielded various technological evolutions [1], and users have needed to collect and select information according to their purposes. In such a situation, data structures called ontologies that organize knowledge through a hierarchical structure, as shown in Figure 1, have received considerable attention, as they are required to systemize the vast amount of data on the Internet. In information science, ontologies are used for geographic information [2] and establishing a general situation awareness framework [3]. However, when manually constructed, an ontology requires a large amount of time and deep knowledge of the target field. Therefore, there is a need for a mechanism to support or substitute ontologies created from unstructured text.

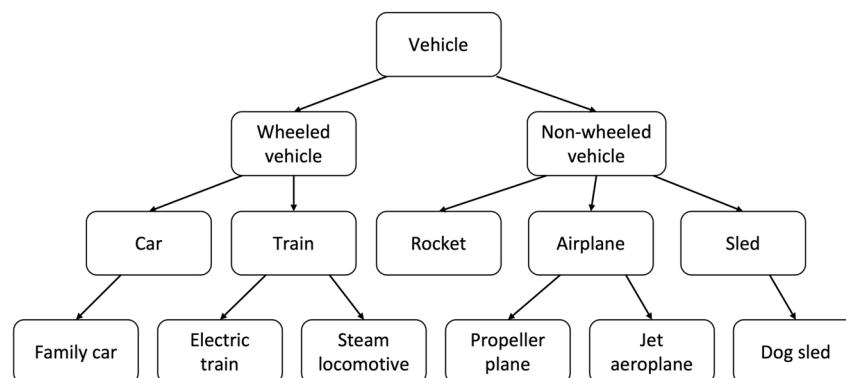


Figure 1. Example of a hierarchical structure of an ontology in which vehicle is the top class.

Automation by machine learning is indispensable with respect to meeting ontology demand, and it is difficult to generate an ontology directly from text via a single methodol-

ogy. An ontology is utilized by unique and sophisticated formats such as ontology web language; it is quite different from typical language-model outputs. Additionally, the required configuration and scale of the ontology depend on the purpose. Therefore, we divided the complex ontology-generation task into the following three subtasks (Figure 2):

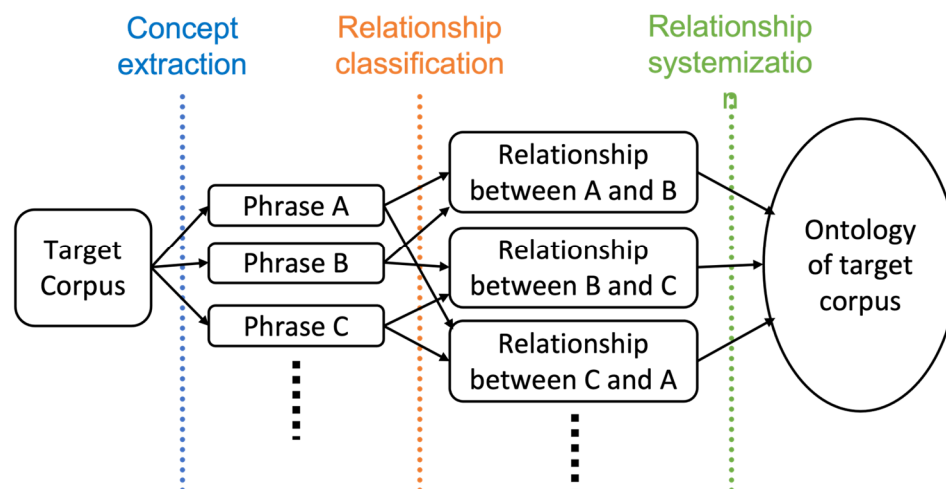


Figure 2. Workflow of ontology generation.

1. Extracting key phrases from a target corpus;
2. Generating a taxonomic structure consisting of a hypernym–hyponym relationship from the extracted phrase set;
3. Creating detailed relationships between phrases according to the intended use of the ontology.

In this research, we focus on the automation of (2) and some cases of (3). Task (1) is a general problem in natural language processing (NLP), and sophisticated methods already exist [4,5]. We can utilize those methods' ideas for our task. However, (2) and (3) are specific to this theme, and there is much room for research. Furthermore, these can be replaced with classification problems frequently used in machine learning.

As a classification method of phrase-pair relationships, a model combining a recurrent neural network (RNN) and word embedding has been proposed in previous research [6]. The overall architecture is based on a traditional problem in the natural language processing (NLP) model, and its input part is remodeled to twin-text reader units to handle pairs of phrases. This method efficiently classifies phrase-pair relationships, especially from the perspective of memory usage and calculation cost. Because of the rapid development of language models, we can improve the method. We describe two major problems with the current method in the next section.

The first problem is handling out-of-vocabulary (OOV) words. In the existing method [7,8], each word is encoded into a feature vector by pretrained vector representation before being entered into the model. At that time, if there are some OOV words in phrases, the zero-vector space is set. For this reason, a large number of vocabulary terms and their vectors are required to improve model versatility; however, as the number of vocabulary terms increases, a larger memory is required. In particular, in tasks focused on specialized corpora, such as ontology generation, rare domain-specific words appear frequently, and the OOV problem is directly linked to model performance. Besides the pretrained model, we apply our model to generate an example of ontology for a question-answering (QA) dataset, such as The Stanford Question Answering Dataset (SQuAD v2.0).

The second problem is updating the text-processing architecture. Context information means additional information generated by word order and combination. The previous method [6] uses RNN to handle context information; however, transformer architecture [9,10] becomes standard in NLP. Many researchers have published papers using transformer

architecture and have shown significant progress. Empirically, we propose that using transformer architecture also yields improvement in our ontology-generation area. In conclusion, our subjects handle OOV words and architecture updates.

Our contribution can be summarized as follows. To improve the limitations of our previous approach, we propose applying a pretrained language model (PLM), such as BERT [9], to taxonomically structure generation tasks. Recently, with the advent of BERT, transfer learning has become a popular trend for language models, enabling extremely high performance by simply finetuning the target downstream tasks. Although various language processing techniques are used in this method, we focus on byte pair encoding (BPE) and transformer architecture. BPE is proposed as a simple data compression technique and is also effective for NLP's subword tokenization [11]. Because BERT posted state-of-the-art scores in various tasks using transformers [12] as their core structure, many language models after BERT have adopted the transformer architecture at their base, also posting state-of-the-art results. Additionally, it is expected that the processing units have high processing power for context information. From these features, we expect significant improvements both experimentally and functionally. We also illustrate a real example to apply our ontology classifier by pretrained language model to a QA dataset.

2. Related Work

2.1. Lexico-Syntactic-Based Ontology Generation

The lexical-syntactic approach to extract the taxonomic relations of words is a famous traditional method of ontology generation [13]. The method uses the position of words and specific fixed phrase patterns in text, analyzes patterns scattered in sentences, and explores the taxonomical relationships available to the ontology. Combined with machine learning, it can help handle exceptional syntax patterns and improve accuracy.

2.2. Word Vectorization-Based Ontology Generation

As another method, the information amount and vectorization representation of words has been used for taxonomic-relation classification in ontologies. This method solves problems that do not have super/sub relationships in a lexico-syntactic approach. The method handles words with numerical representation trained by word position in a sentence, and the relationships between them are determined using a machine-learning model that takes a vector as input. The representative example of this approach is a combination of word embedding and support-vector machine learning [14].

2.3. RNN-Based Ontology Generation

RNN-based models were the first to solve the task of classifying relations between phrase pairs using grammatical semantic interpretation [6]. Prior to this model, models that combined word embedding and a simple classifier for ontology generation were the mainstream, and the disadvantage was that complex terms consisting of multiple words could not be compared. By using a neural network with an RNN-based recurrent structure, it is now possible to classify compound words. This model follows the structure of the traditional sequential language model, and it is characterized by having two input layers for handling two input sentences. This model is roughly divided into four parts (embedding layer, RNN cells, concatenation process, and classification layers). Those processes are calculated in order.

2.4. Hypernym and Synonym Matching for BERT

Hypernym and synonym matching using FinBERT [15] and SBERT [16] finetuning by data augmentation of financial terms shows improved performance.

Our system has pretrained the bidirectional encoder representations from transformers (BERT) [12] and A Lite BERT (ALBERT) [17] from the scratch by Wordnet data and showed far better performance.

2.5. Ontology Generation Using the Framework

Building a knowledge graph from scratch requires training machine learning models on a huge number of datasets, in addition to powerful NLP techniques and inference capabilities. In this research [18], they solved this problem by using a third-party framework. In addition to that, they have performed data cleaning on the unstructured corpus and error checking on the KGs generated using YAGO [19] and DOLCE [20].

3. Preliminaries

3.1. Relationship Classification between Phrase Pairs

To generate an ontology, it is necessary to automate the task of classifying relationships between phrases. The central element of an ontology is the hierarchical structure of concepts, but it is difficult to build the hierarchical structure all at once, and it is necessary to break down machine learning into easy-to-implement tasks. Therefore, in ontology generation, relationships between concepts (such as synonyms and hypernyms) are identified using a classification model. It then builds a hierarchical structure by organizing the acquired relationships. If the concept is interpreted as a set of phrases consisting of multiple words, it is possible to substitute it with a phrase instead of a concept. To summarize, ontology generation can be downsized into a classification problem that takes two phrases as the input and outputs the relationship between them.

To process two input-text-data and output-relation labels, there are four essential factors in classifying relationships between concepts.

- **Word embedding:** By converting words into their corresponding vectors, they are converted into a format that can be easily handled by neural networks. Additionally, because ontology generation often needs to deal with rare words that are task-specific, learning those in advance in a large corpus is preferable.
- **Acquisition of context information:** Phrases used in ontology generation often consist of a small number of words. However, the connection between words is stronger than between general sentences, and it is necessary to process contextual information.
- **Concatenation:** because the input data for this task comprise two independent phrases, we need to combine the information at some stage in the process.
- **Classifier:** we apply the information obtained in the presented steps to the classification model and calculate the final output label.

3.2. Encoding and Architecture for a Pretrained Model

3.2.1. Byte Pair Encoding

As mentioned in the previous section, BPE is basically proposed as a data compression technique and is recognized for its versatility in the field of NLP [21–23]. This method is a variable-length encoding that displays text as a series of symbols and repeatedly merges the most frequent symbol pairs into a new symbol. Compared with regular word-based methods, the number of words and associated vectors in the dictionary can be significantly reduced while maintaining expressiveness. Words that do not exist in this dictionary can be expressed by subwords.

3.2.2. BERT

BERT [12] is an architecture of a multiple-layered transformer array that trains deep, bidirectional representations from unlabeled texts. It is pretrained from a large unsupervised text corpus, such as Wikipedia, using the two methods of “masked word prediction” and “next sentence prediction.”

Unlike traditional sequential or recurrent models, the attention architecture processes the whole input sequence at once, enabling all input tokens to be processed in parallel. The layers of BERT are based on transformer architecture. The pretrained BERT model can be finetuned with just one additional layer to obtain state-of-the-art results in a wide range of NLP tasks.

3.2.3. ALBERT

ALBERT [17] is a next-generation BERT-based architecture with an optimized configuration. The basic structure of ALBERT is the same as BERT. However, ALBERT reduces the parameters of the embedding layer and shares the transformer layers, improving the memory usage and the calculation time. ALBERT can use more layers and a larger hidden size if the same computational resources are available, allowing for a more complex and expressive model than BERT.

3.3. Noun Phrase Extraction

Syntactic Parsing

Parsing is an NLP task that extracts the dependency of a sentence by analyzing the grammatical structure and word relationships. Unlike programming languages, NLP grammars have more than one syntax tree that can be interpreted for a single sentence. Therefore, it is necessary to choose the best of multiple syntax trees. There are two types of parsing: deep learning and rule-based. With rule-based analysis, the syntax is checked in the order in which it is defined, meaning that it is often not optimal. However, dependency parsing using a deep-learning model compares several patterns for a sentence and chooses the best one. The model can be expected to provide a better result than rule-based analysis. For this purpose, we use the state-of-the-art model [24] from “Penn Treebank” [25].

4. Ontology Classifier Using PLM

4.1. Learning Procedure

The proposed PLM-based method uses fully divided two-stage learning: general learning and task-specific learning. In general learning, PLM is trained with tasks that require only an unlabeled corpus, making it possible to learn a large-scale corpus such as Wikipedia data. PLM acquires general and broad linguistic knowledge consisting of a large amount of sentence information. In task-specific learning, additional layers are appended at the bottom of the model for converting the output format. Then, the model is finetuned with labeled task-specific data. This research refers to the learning procedure, and we finetune PLM to taxonomic relation classification tasks.

This research utilizes a Hugging Face transformer library [26] implemented in Python. The library provides the implementation of various language models and model data that have acquired prior knowledge through the general learning process. It facilitates model comparison by collecting the latest models and generic pretraining models under a unified API. In this paper, we utilize this library to implement pretraining and general learning of the model published by google research.

4.2. Architecture

We use a simple architecture that appends the preprocessing unit, dropout regularization, and a softmax classification layer at each end of the language model. This is a standard configuration of PLMs when they are applied in classification tasks, indicating that the proposed method does not require a task-specific architecture. Figure 3 shows the overall architecture of our model in a task-specific ontology-generation stage, and it consists of the following three main stages. In this section, we describe the details and the intention of each part individually.

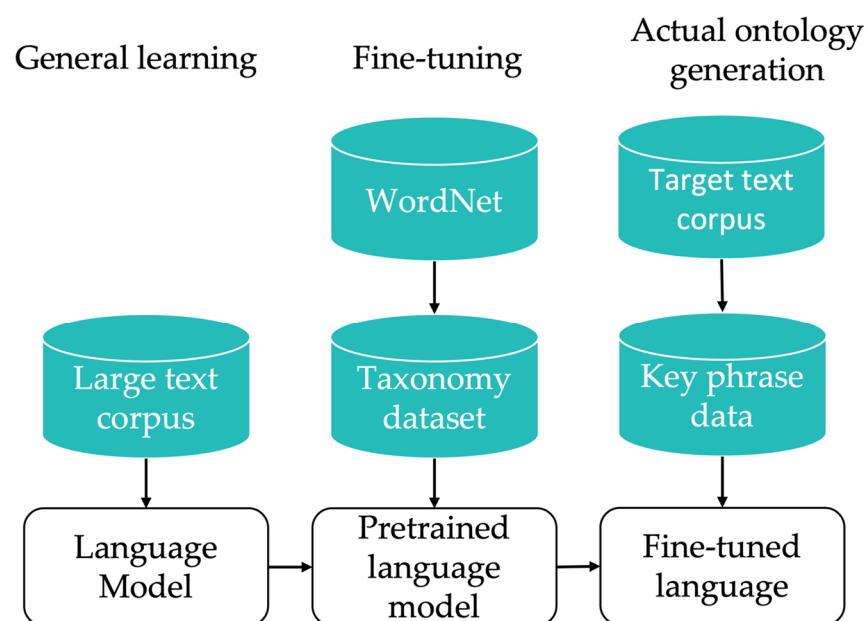


Figure 3. The learning procedure of the proposed pretrained language model-based method.

4.2.1. Preprocess

The first stage of our architecture is preprocessing. In most cases, PLMs require a specific format of the input sentences to implement subword expressions and accept inputs for various tasks. Furthermore, relationship-classification tasks require a pair of phrases as input data, and we perform the following preprocessing steps on the phrase pairs before entering them into the model (an example of these steps is shown in Table 1).

1. Concatenation and Special Token Insertion: First, the input phrase pair is concatenated into one sentence. At this time, a classifier token (CLS) is inserted at the front of the first phrase and separator tokens (SEP) [12] are appended in the middle of the two phrases and at the end of the second phrase.
2. Tokenization: The concatenated phrases are divided into subwords by a tokenizer corresponding to each language model. The number of divided subwords is equal to or greater than the number of words included in the phrase.

Table 1. The architecture of the proposed pretraining-based method for taxonomic structure generation and the calculation process flow.

	Phrase A	Phrase B
Input	Airplane	Jet aeroplane
Concatenation	[CLS] airplane [SEP] jet aeroplane [SEP]	
Tokenization	[CLS] airplane [SEP] jet aero ##plane [SEP]	

4.2.2. Pretrained Language Model

In this part, calculations for task-specific learning and actual ontology generation of some PLMs are performed (Figure 4). The number of outputs of PLMs corresponds to the number of input sequences. Because this study is a classification problem, we extract only the head vector corresponding to the [CLS] token inserted in the preprocessing.

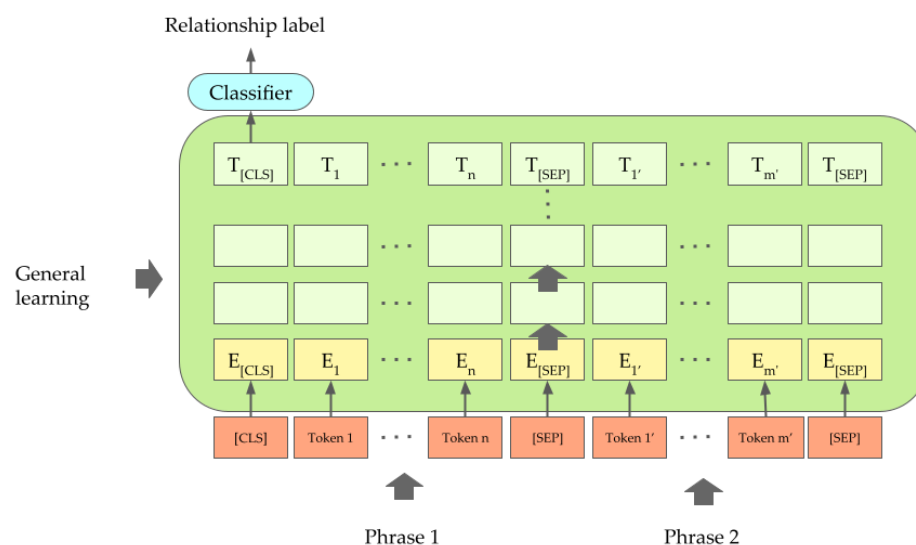


Figure 4. The architecture of the proposed pretraining-based method for taxonomic structure generation and the calculation process flow.

4.2.3. Classification Layers

This part classifies hidden vectors (consisting of two-phrase relationship information) into four types of relationship labels. First, a single linear layer translates into as many vectors as class labels. Then, dropout with a probability factor of 0.1 is applied to regularize and prevent overfitting, and the dropout is only applied in the training phase and not in the inference phase. Finally, the softmax classification layer outputs the probabilities of the input text belonging to each of the class labels, such that the sum of the probabilities is 1. The softmax layer is just a fully connected neural network layer with the softmax activation function.

5. Data Collection

5.1. Phrase-Pair Relationship Datasets

As described in Section 2.1, ontology generation can be realized by relationship classification between phrases. The proposed method using PLMs performs pretraining in the general learning process, and fewer data are required for task-specific learning than previous methods, though a large-scale dataset is still required for general ontology generation. Because the ultimate goal is to build an ontology, it is ideal to obtain it from a real ontology or something close to it. Therefore, in this study, we acquired data from WordNet [27], a large-scale concept dictionary and database that has a structure similar to that of an ontology. Then, as a test for extracting ontologies from real data, we extracted noun phrases contained in sentences from SQuAD data. The rest of this chapter provides an overview of datasets and how to acquire phrase-relationship datasets.

5.2. Overview of WordNet

In WordNet, to combine concepts that can be expressed in different notations with the same meaning into one object, a group of phrases called a synset is used as the smallest unit. All synsets on WordNet form a hierarchical structure; if synset A is a more abstract version of synset B, synset A is defined as a hypernym of synset B. From the structure, WordNet can be viewed as a huge, multipurpose ontology that covers the entire language, and it can be used as teacher data in ontology generation.

5.3. Dataset Extraction from WordNet

In this research, we extracted pairs of phrases and the four kinds of relationships (synonym, hypernym–hyponym, hyponym–hypernym, and unrelated) from the taxonomy structure of WordNet (Figure 5). The details of the datasets are shown in Table 2. First, for

all noun synsets, all pairs of terms are registered in one synset as synonym pairs. Next, we extract hyponyms based on all target-noun synsets and register them as hypernym–hyponym relations. Moreover, pairs of phrases where the order is reversed are labeled as hyponym–hypernym relations. Finally, we made many pairs of phrases randomly that do not have special relationships and labeled them as unrelated pairs.

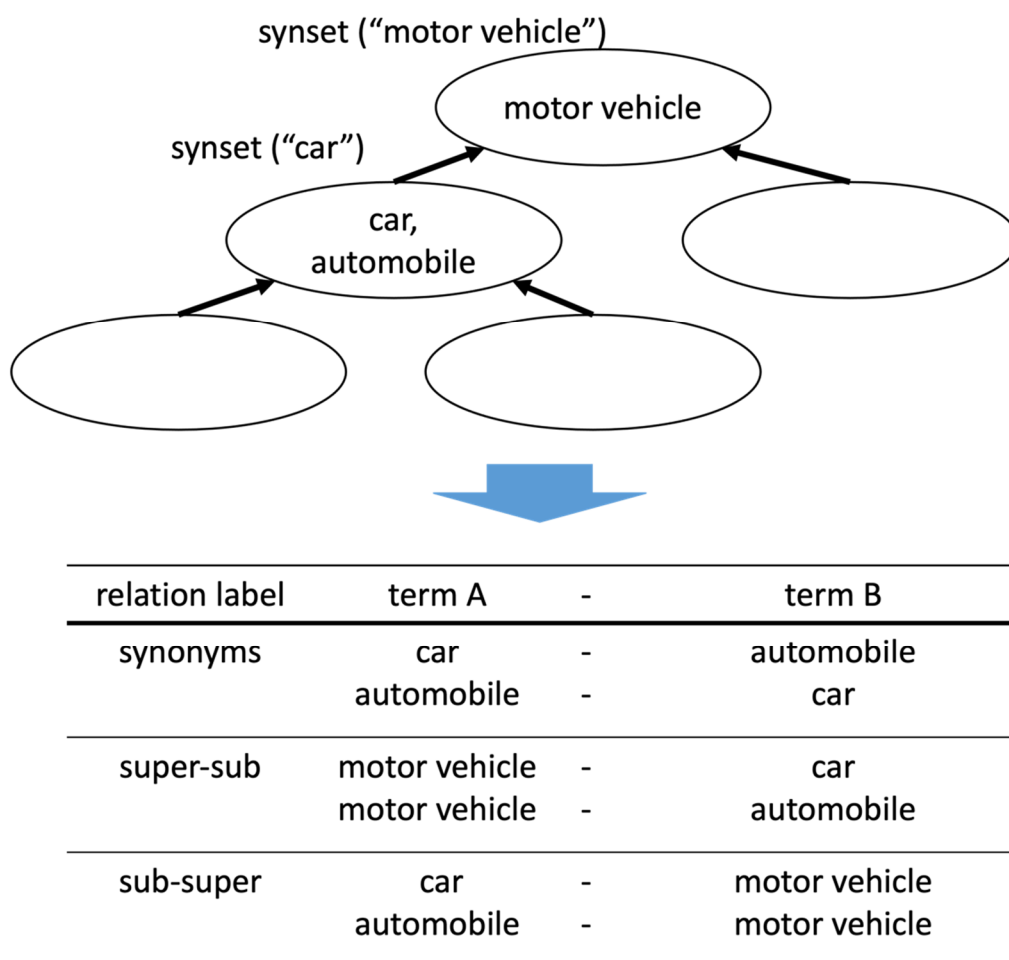


Figure 5. An example of the data extraction process from WordNet’s taxonomic structure.

Table 2. Details of dataset extracted from WordNet.

Relation Label	Number of Data
Synonym	135,658
Hypernym	215,554
Hyponym	215,554
Unrelated	500,000
Total	1,066,766

5.4. Dataset of SQuAD V2.0

The SQuADv2.0 dataset gives context and multiple questions. The answer to a question returns the location of the first and last letters, or if the answer to that question is not in the context, it returns that it cannot be answered [28]. This has resulted in a dataset that confirms the accuracy of the QA system and is more realistic. These datasets focus on getting closer to the actual QA situation. A QA system uses ontology relationships when questions and answers are created. Therefore, it is possible to extract the ontology from the QA dataset with our model.

5.5. Dataset Extraction from SQuAD V2.0 for Ontology Generation

5.5.1. Extraction of Nouns

In this research, we use a part-of-speech (POS) tagger [29] to extract the nouns before and after a particular word (is, are, was, were, 's, 're, such as). First, we look for sentences that have a particular word. For each sentence in each context, we use the POS tagger to extract the nouns from the sentence. Then, we divide the words into two groups: those before a particular word and those are after it. All combinations of words in the two groups are considered as candidates.

5.5.2. Extraction of Noun Phrases

In this study, we use a model based on encoder-decoder model using label-attention-base with improved self-attention [24] to analyze sentences and extract none phrases. We apply the model to each sentence in each context. This allows us to recognize the sentence structure better and extract the noun phrases from it. Then, all patterns of noun phrases in each context are considered as candidates.

6. Experiment

6.1. Training and Validation

In the experiment, we quantitatively compared the RNN-based and PLM-based methods by observing the classification accuracy. To compare the BPE with the RNN method using the existing Word2vec, we investigated the combination of the embedding layer of the BERT-based model and RNN. Using the dataset acquired from WordNet described in Table 3, we trained the model and measured the classification performance after task-specific learning. At this time, the dataset of WordNet was randomly divided into three types: training data, validation data, and test data. Validation and test datasets each have 10,000 data, and all other data are used for training.

Table 3. Details of distribution of the dataset.

	Synonym	Hypernym	Hyponym	Unrelated
Train	133,164	211,514	211,445	490,643
Validation	1233	2002	2108	4657
Test	1261	2038	2001	4700
Total	135,658	215,554	215,554	500,000

6.2. Model Setup

To confirm the validity of PLMs for the relationship-classification task, it is necessary to perform validation on many models. Additionally, these language models have some configurable parameters, such as the number of layers and hidden size. Similar to neural network models in other domains, verification of the effects of these parameters is also an important factor for PLMs. For these reasons, we compared the major PLMs architectures, and their configurations are summarized in Table 4. The vocabulary size of word-embedding methods is illustrated in Table 5.

Table 4. Configuration of pretrained language models used for comparison and learning parameters.

	BERT		ALBERT	
	Base	Large	Base	Large
Optimizer	Adam			
Learning rate	1×10^{-5}			
Transformer layers	12	24	12	24
Hidden Size	768	1024	768	1024
Embedding Size	768	1024	128	128
Parameters	108 M	334 M	12 M	18 M

Table 5. Vocabulary size of word-embedding methods.

	Vocabulary Size
Word2vec (previous)	297,141
BERT-Embedding (Subworld Representation)	30,522

6.3. Ontology Generation for the Real Text

As an ontology generation test, we use the model trained on WordNet to classify the relationship between nouns and noun phrases in a SQuAD v2.0 context. In this test, we use the model that performed best on the WordNet classification task. We use this model to determine the relationship in each SQuAD v2.0 context and confirm what kind of results were obtained.

7. Evaluation

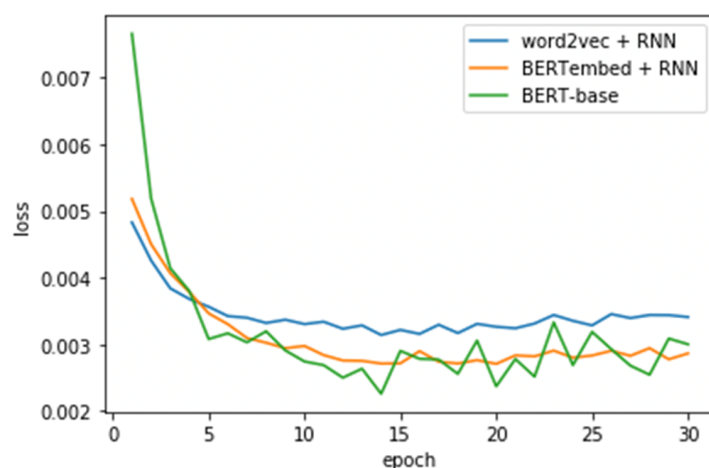
7.1. Comparison of Accuracy

In this experiment, we compared the accuracy of the previous model, proposed model, and variation. These models were trained for any number of epochs by phrase-pair relationship datasets extracted from WordNet. Then, this learning process was stopped based on the validation results after empirical measurement. Finally, the classification performance was evaluated by the test datasets. The accuracy results are summarized in Table 6. The calculation times required for learning one epoch were also compared, as also shown in Table 6.

Table 6. Comparison of relationship-classification accuracies between phrases, with the calculation time for learning expressed as a ratio when the learning time of RNN is set to 1.0.

	Accuracy (Four Classes)	Recall	Precision	F1	Ratio of Calculation Time
Word2vec + RNN (Previous)	87.1%	94.87	93.13	93.88	1.00
BERT-Embedding + RNN	89.6%	95.89	93.4	94.96	1.21
BERT-Base	98.1%	98.8	99	98.9	6.26
BERT-Large	98.6%	99.7	99.4	99.55	15.34
ALBERT-Base	96.8%	98.6	99.1	98.9	4.23
ALBERT-Large	98.3%	99.3	99.05	99.17	12.01

Figures 6 and 7 show that the PLM-based method can achieve better performance than RNN, even in small epochs.

**Figure 6.** Comparison of validation loss transitions.

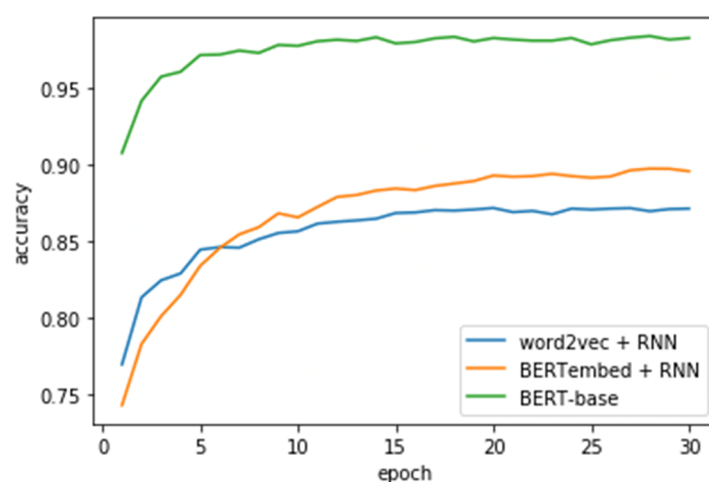


Figure 7. Comparison of validation accuracy transitions.

This experiment shows that the proposed PLM-based method has significantly better accuracy than existing methods. In particular, BERT-large reaches the highest accuracy of 98.6% among the models used for comparison. By contrast, the calculation time is much longer than the existing methods, and the calculation time of the minimum configuration ALBERT-based is four times that of the RNN-based method. In comparison with the embedding method, the BERT-embed + RNN method has better accuracy while keeping smaller vocabularies than the existing Word2vec + RNN method, indicating that BPE is an effective method for ontology generation.

7.2. Comparison of Batch Size with BERT-Base

In this section, we performed additional experiments regarding batch size with the BERT-based method (Figure 8). Typically, PLM has a large number of stacked transformer layers to acquire linguistic knowledge that can support a wide variety of tasks through general learning; the bulk of this model comes at the expense of memory usage and computation time, and many PLMs are more expensive than regular language models, as shown in previous experiments. We have a batch-size limit, and in particular, large models need to be trained with a rather small batch. In previous experiments, we used a maximum batch size on each architecture because of the memory-usage limit of our machine. However, the effect of batch size on calculation speed is unknown, and we show the effect of changes in batch size on calculation time. As a result, the memory occupation ratio for each piece of data increases, and the batch size tends to be limited. This theory examines the dependence of the proposed method on the learning environment by measuring the effect of batch size on the model.

7.3. Ontology-Generation Experiment Results

In this section, we carried out the process to generate ontology for SQuAD v2.0 in Figure 9. We show the results of extracting taxonomical relationships from the actual data taken from the SQuAD dataset. As seen Table 7, for noun pairs, the model can reduce the number of phrases from over 1 million to 250,000, and for noun phrase pairs, the model was able to reduce the number of phrases from nearly 3 million to 560,000. The model has been trained from scratch using sentences of the SQuAD v2.0 dataset, and terms or term sequences for taxonomy classification have been selected by a sentence parser. We can remove unnecessary terms with the parser for term filtering. As there are several hundred topic categories in the SQuAD dataset, we show the generation result of a small set of the topics, such as art, music, iPad, etc.

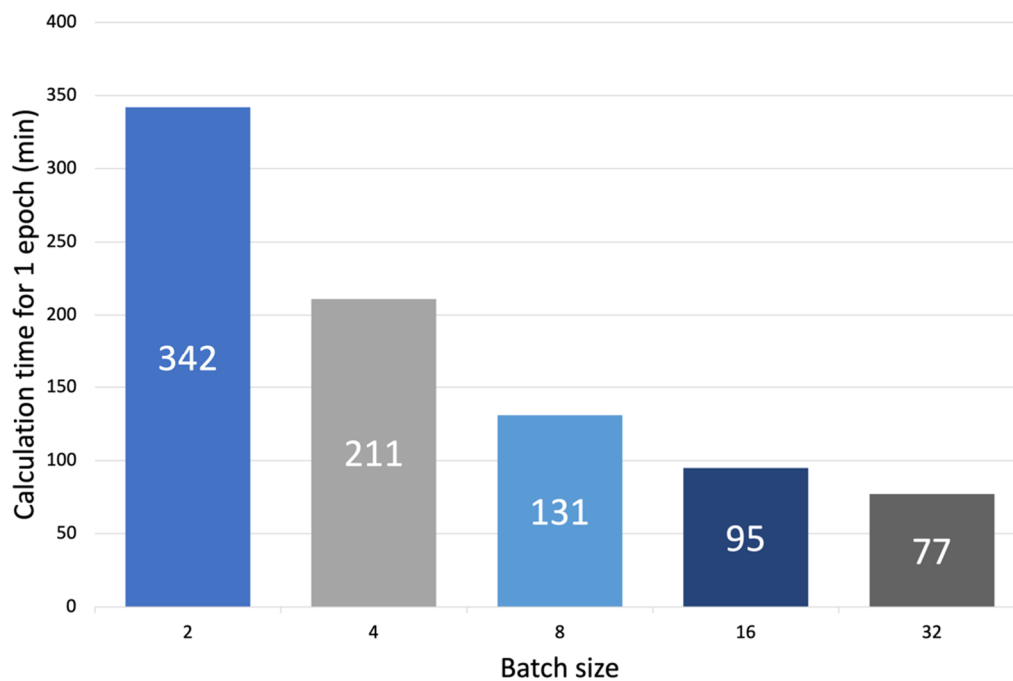


Figure 8. Effect of train batch size on calculation time with a BERT-based model.

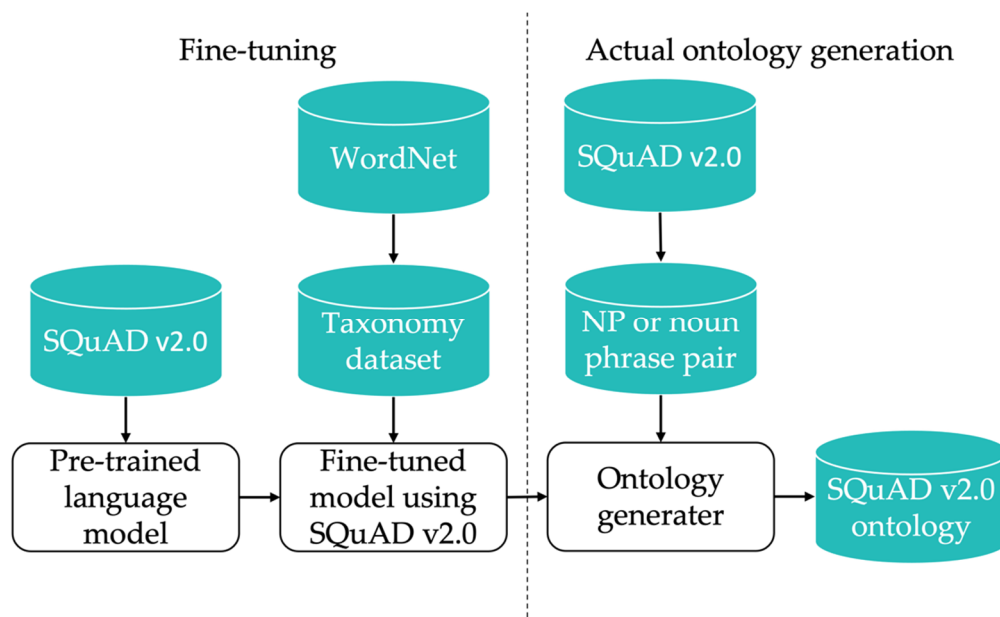


Figure 9. The procedure to generate ontology for SQuAD v2.0.

Table 7. The total number of noun pairs and noun phrases extracted from SQuAD v2.0 and the total number of ontology relations generated by the ontology generator.

	Number of Extracted Pairs	Number of Distilled Pairs
Noun pairs	1,106,386	426,450
Noun phrase pairs	2809,133	280,397

Firstly, we got 47 correct taxonomical (super-sub/sub-super) classifications of 50 noun pairs generated by human evaluation (Table 8). The classifications are reasonable and reflect facts in real-life or conceptual hierarchy. Examples of misclassified pairs include:

'department'-'store' (it may come from *'department store'-'store'*), *'passages'-'passages'* (classified as sup-sub, but it is the synonym), *'environment'-'reputation'* (may come from *'environment reputation'-'reputation'*). Secondly, there are two cases that cannot be counted as correct taxonomical relations of 36 noun phrase cases (Table 9): *'official posts'-'official posts'* (synonym) and *'succession important posts'-'hereditary positions'* (good for having the same meaning but difficult to be complete hypernym). Bypassing the preprocessed text to the model pretrained Wordnet, we were able to extract the pairs whose relationship could be confirmed by humans. This procedure made it possible to provide an ontology set from the text. However, the results require fine-grained filtering for generating taxonomical pairs.

Table 8. The result of applying the ontology generator to the extracted noun pairs.

Noun 1	Noun 2	Relationship
Beyoncé	actress	sub-sup
Beyoncé	artist	sub-sup
music	pop	sup-sub
Grammy	award	sub-sup
Beyoncé	singer	sub-sup
family	parents	sup-sub
B'Day	birthday	synonyms
female	Beyoncé	sup-sub
singer	Swift	sup-sub
career	life	sub-sup
announcement	tweets	sup-sub
Barbara	female	sub-sup
artist	entertainer	sub-sup
star	performer	sub-sup
choreography	dance	sub-sup
video	YouTube	sup-sub
video	parodies	sup-sub
albums	music	sub-sup
records	music	sub-sup
March	music	sub-sup
service	Spotify	sup-sub
service	industry	sub-sup
women	grandmother	sup-sub
department	stores	sup-sub
mother	human	sub-sup
Chopin	composer	sub-sup
Chopin	pianist	sub-sup
era	generation	sup-sub
birthdate	date	sub-sup
passages	passages	sup-sub
commenting	piano-bashing	sup-sub
student	role	sub-sup
Liszt	musician	sub-sup
friendship	relationship	sub-sup
rift	relationship	sub-sup
woman	daughter	sup-sub
woman	mother	sup-sub
couple	people	sub-sup
apartment	accommodation	sub-sup
canon	music	sub-sup
preludes	music	sub-sup
sonata	music	sub-sup
method	technique	sup-sub
rubato	melody	sub-sup
environment	reputation	sup-sub

Table 8. Cont.

Noun 1	Noun 2	Relationship
dynasty	leaders	sub-sup
suzerainty	region	sub-sup
Tibet	China	sub-sup
ethnicities	Han	sup-sub
King	Emperor	synonyms

Table 9. Result of applying the ontology generator to the extracted noun phrases.

Noun Phrase 1	Noun Phrase 2	Relationship
official posts	official posts	sup-sub
succession important posts	hereditary positions	sup-sub
succession important posts	official posts	sub-sup
true Han representatives	Han Chinese government	synonyms
1390	14th century	sub-sup
shamanistic ways	native Mongol practices shamanism blood sacrifice	sup-sub
event	conflict	sup-sub
event	war	sup-sub
aid gelug monks supporters	help	sub-sup
gelug monasteries	traditional religious sites	sub-sup
fifth Dalai Lama lozang gyatso	Dalai Lama	sub-sup
Chinese claims suzerainty Tibet	territory	sub-sup
portable media players multipurpose pocket computers	iPod	sup-sub
128 gb iPod touch	iPods	sub-sup
iPods	digital music players	sub-sup
product	iPod	sup-sub
fonts	Chicago font	sup-sub
commercial use	trademark	sup-sub
100 db	maximum volume output level	synonyms
legal limit	user-configurable volume limit	sup-sub
legal limit	maximum volume output level	sup-sub
maximum volume output level	user-configurable volume limit	synonyms
implementation interface	dock connector	sup-sub
apple lightning cables	new 8pin dock connector named lightning	synonyms
cars	BMW	sup-sub
cars	Volkswagen	sup-sub
advanced menu iTunes	iPod software	sub-sup
alternative opensource audio formats ogg vorbis flac	several audio file formats	sub-sup
audio files	midi files	sup-sub
audio files	mpeg4 QuickTime video formats	sup-sub
audio files	several audio file formats	synonyms
iPods library	entire music libraries music playlists	synonyms
iPods library	iTunes library	synonyms
main computer library	entire music libraries music playlists	synonyms
devices	iPhone	sup-sub
devices	iPod touch	sup-sub

8. Conclusions and Future Work

In this paper, we applied the procedure of transfer learning and PLM technologies to ontology generation to improve the performance of relationship classification between phrase pairs. As a result, a dramatic improvement of +12.4% was achieved in terms of accuracy without sophisticated architecture. The results also show that large-model configurations provide better accuracy in PLM-based approaches. However, in terms of computational resources, the results show that ALBERT, a model with improved memory usage, fell short of existing methods. Combining learning processes such as DistilBERT [30] and TinyBERT [31] with knowledge-distillation and optimizing models such as ALBERT to

solve these weaknesses is an example of future work. Although the PLM-based method has succeeded in reproducing the hierarchical structure of general-purpose ontologies such as WordNet, it does not support domain-specific ontologies. In fact, we were able to extract an ontology by using SQuAD v2.0 for noun phrases and nouns and found that our model mostly works. However, as there are some misclassified terms, we should carry out further investigation to develop a method for a better ontology. Therefore, creating an ontology that depends on purpose, quality, and uniqueness is also one of the issues to be addressed in the future. The codes are available at (<https://github.com/atshb/Text2ontology>) (accessed on 25 October 2021).

Author Contributions: Conceptualization, I.P.; methodology, A.K., A.O. and R.S.; software, A.K. and A.O.; validation, A.O. and I.P.; formal analysis, A.K., A.O. and I.P.; investigation, A.K., A.O. and R.S.; resources, I.P.; data curation, A.K. and A.O.; writing—original draft preparation, A.K., A.O. and R.S.; writing—review and editing, A.K. and I.P.; visualization, A.K. and A.O.; supervision, I.P.; project administration, I.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Acknowledgments: The authors are thankful for the dedicated effort of all the reviewers who contributed to the reviewing process. Also, our gratitude goes to the editorial board of MDPI Electronics journal for giving us the opportunity to publish this paper, and to the Electronics editorial office staff for their hard and precise work.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Oba, A.; Paik, I.; Kuwana, A. Automatic Classification for Ontology Generation by Pretrained Language Model. In Proceedings of the International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems; Artificial Intelligence Practices. Fujita, H., Selamat, A., Lin, J.C.-W., Ali, M., Eds.; Springer International Publishing: Cham, Switzerland, 2021; pp. 210–221.
2. Bittner, T.; Donnelly, M.; Smith, B. A spatio-temporal ontology for geographic information integration. *Int. J. Geogr. Inf. Sci.* **2009**, *23*, 765–798. [[CrossRef](#)]
3. Paik, I.; Komiya, R.; Ryu, K. Customizable active situation awareness framework based on meta-process in ontology. In Proceedings of the International Conference on Awareness Science and Technology (iCAST) 2013, Fukushima, Japan, 2–4 November 2013.
4. Zhu, H.; Paschalidis, I.C.; Tahmasebi, A. Clinical concept extraction with contextual word embedding. *arXiv* **2018**, arXiv:1810.10566.
5. Brack, A.; D’Souza, J.; Hoppe, A.; Auer, S.; Ewerth, R. Domain-independent extraction of scientific concepts from research articles. *Adv. Inf. Retr.* **2020**, *12035*, 251–266.
6. Oba, A.; Paik, I. Extraction of taxonomic relation of complex terms by recurrent neural network. In Proceedings of the 2019 IEEE International Conference on Cognitive Computing (ICCC), Milan, Italy, 8–13 July 2019; pp. 70–72.
7. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient Estimation of Word Representations in Vector Space. *arXiv* **2013**, arXiv:1301.3781.
8. Pennington, J.; Socher, R.; Manning, C. GloVe: Global Vectors for Word Representation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 25–29 October 2014; pp. 1532–1543.
9. Duan, S.; Zhao, H. Attention is all you need for Chinese word segmentation. *arXiv* **2020**, arXiv:1910.14537.
10. Dowdell, T.; Zhang, H. Is attention all what you need?—An empirical investigation on convolution-based active memory and self-attention. *arXiv* **2019**, arXiv:1912.11959.
11. García, I.; Agerri, R.; Rigau, G. A common semantic space for monolingual and cross-lingual meta-embeddings. *arXiv* **2021**, arXiv:2001.06381.
12. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2018**, arXiv:1810.04805.
13. Klausner, C.; Zhekova, D. Lexico-syntactic patterns for automatic ontology building. In Proceedings of the Second Student Research Workshop Associated with RANLP 2011, Hissar, Bulgaria, 13 September 2011; pp. 109–114. Available online: <https://www.aclweb.org/anthology/R11-2017/> (accessed on 25 October 2021).
14. Omine, K.; Paik, I. Classification of taxonomic relations by word embedding and wedge product. In Proceedings of the 2018 IEEE International Conference on Cognitive Computing (ICCC), San Francisco, CA, USA, 2–7 July 2018; pp. 122–125.
15. Araci, D. FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models. *arXiv* **2019**, arXiv:1908.10063.
16. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. *arXiv* **2019**, arXiv:1908.10084.

17. Lan, Z.; Chen, M.; Goodman, S.; Gimpel, K.; Sharma, P.; Soricut, R. ALBERT: A lite BERT for self-supervised learning of language representations. *arXiv* **2020**, arXiv:1909.11942.
18. Elnagar, S.; Yoon, V.; Thomas, M.A. An Automatic Ontology Generation Framework with An Organizational Perspective. In Proceedings of the Hawaii International Conference on System Sciences, Grand Wailea, Hawaii, 1 July–1 October 2013. Available online: https://aisel.aisnet.org/hicss-53/ks/knowledge_flows/3/ (accessed on 25 October 2021).
19. Wang, Y.; Zhu, M.; Qu, L.; Spaniol, M.; Weikum, G. Timely YAGO: Harvesting, Querying, and Visualizing Temporal Knowledge from Wikipedia. In Proceedings of the 13th International Conference on Extending Database Technology, New York, NY, USA, 23–26 March 2010; pp. 697–700.
20. Gangemi, A.; Guarino, N.; Masolo, C.; Oltramari, A. Sweetening WORDNET with DOLCE. *AI Mag.* **2003**, *24*, 13. [[CrossRef](#)]
21. Sennrich, R.; Haddow, B.; Birch, A. Neural machine translation of rare words with subword units. *arXiv* **2016**, arXiv:1508.07909.
22. Bojanowski, P.; Grave, E.; Joulin, A.; Mikolov, T. Enriching word vectors with subword information. *Trans. Assoc. Comput. Linguist.* **2017**, *5*, 135–146. [[CrossRef](#)]
23. Heinzerling, B.; Strube, M. Bpemb: Tokenization-free pre-trained sub-word embeddings in 275 languages. *arXiv* **2017**, arXiv:1710.02187.
24. Mrini, K.; Derroncourt, F.; Tran, Q.; Bui, T.; Chang, W.; Nakashole, N. Rethinking Self-Attention: Towards Interpretability in Neural Parsing. *arXiv* **2020**, arXiv:1911.03875.
25. Marcus, M.P.; Marcinkiewicz, M.A.; Santorini, B. Building a Large Annotated Corpus of English: The Penn Treebank. *Comput. Linguist.* **1993**, *19*, 313–330. Available online: https://repository.upenn.edu/cis_reports/237/ (accessed on 25 October 2021).
26. Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv* **2020**, arXiv:1910.03771.
27. Miller, G.A.; Beckwith, R.; Fellbaum, C.; Gross, D.; Miller, K.J. Introduction to WordNet: An On-line Lexical Database. *Int. J. Lexicogr.* **1990**, *3*, 235–244. [[CrossRef](#)]
28. Rajpurkar, P.; Jia, R.; Liang, P. Know what you don’t know: Unanswerable questions for SQuAD. *arXiv* **2018**, arXiv:1806.03822.
29. Toutanova, K.; Klein, D.; Manning, C.D.; Singer, Y. Feature-Rich Part-of-Speech Tagging with a Cyclic Dependency Network. In Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology—NAACL’03, Edmonton, AB, Canada, 27 May–1 June 2003; Volume 1, pp. 173–180.
30. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. Distilbert, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv* **2020**, arXiv:1909.10351.
31. Jiao, X.; Yin, Y.; Shang, L.; Jiang, X.; Chen, X.; Li, L.; Wang, F.; Liu, Q. TinyBERT: Distilling BERT for natural language understanding. *arXiv* **2020**, arXiv:1909.10351.